



# Python Programming (BCC-302)

## UNIT – 4: Python File Operations

# Content

- Open and Close File
- Reading Files
- Writing in Files
- Reading & Writing Files using With Statement
- Seek() and tell() methods





# Open and Close Files

# Opening a File

Before you can read or write a file, you have to open it using Python's built-in `open()` function. This function creates a **file** object, which would be utilized to call other support methods associated with it.

## Syntax

```
file object = open(file_name [, access_mode][, buffering])
```

Here are parameter details-

- **file\_name:** The `file_name` argument is a string value that contains the name of the file that you want to access.
- **access\_mode:** The `access_mode` determines the mode in which the file has to be opened, i.e., read, write, append, etc.
- A complete list of possible values is given below in the table. This is an optional parameter and the default file access mode is read (r).
- **buffering:** If the buffering value is set to 0, no buffering takes place. If the buffering value is 1, line buffering is performed while accessing a file. If you specify the buffering value as an integer greater than 1, then buffering action is performed with the indicated buffer size. If negative, the buffer size is the system default (default behavior).`open()`



| Modes | Description                                                                                                                                         |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| r     | Opens a file for reading only. The file pointer is placed at the beginning of the file. This is the default mode.                                   |
| rb    | Opens a file for reading only in binary format. The file pointer is placed at the beginning of the file. This is the default mode.                  |
| r+    | Opens a file for both reading and writing. The file pointer placed at the beginning of the file.                                                    |
| rb+   | Opens a file for both reading and writing in binary format. The file pointer placed at the beginning of the file.                                   |
| w     | Opens a file for writing only. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing.                  |
| wb    | Opens a file for writing only in binary format. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing. |



|     |                                                                                                                                                                                                                                           |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| w+  | Opens a file for both writing and reading. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing.                                                                       |
| wb+ | Opens a file for both writing and reading in binary format. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing.                                                      |
| a   | Opens a file for appending. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing.                                        |
| ab  | Opens a file for appending in binary format. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing.                       |
| a+  | Opens a file for both appending and reading. The file pointer is at the end of the file if the file exists. The file opens in the append mode. If the file does not exist, it creates a new file for reading and writing.                 |
| ab+ | Opens a file for both appending and reading in binary format. The filepointer is at the end of the file if the file exists. The file opens in the append mode. If the file does not exist, it creates a new file for reading and writing. |



## The File Object Attributes :

- Once a file is opened and you have one *file* object, you can get various information related to that file.
- Here is a list of all the attributes related to a file object-

| Attribute                | Description                                      |
|--------------------------|--------------------------------------------------|
| <code>file.closed</code> | Returns true if file is closed, false otherwise. |
| <code>file.mode</code>   | Returns access mode with which file was opened.  |
| <code>file.name</code>   | Returns name of the file.                        |



# Closing a File

## The close() Method

- The close() method of a file object flushes any unwritten information and closes the file object, after which no more writing can be done.
- Python automatically closes a file when the reference object of a file is reassigned to another file. It is a good practice to use the close() method to close a file.

## Syntax

```
fileObject.close();
```

**Example:**

```
f=open('sample.txt','r+')  
print(f.name)  
f.close()  
print('file closed')
```

**Output:**

```
Sample.txt  
File closed
```







# Reading Files

# read() Method

The read() method reads a string from an open file. It is important to note that Python strings can have binary data apart from the text data.

## Syntax

- `fileObject.read([count]);`
- Here, passed parameter is the number of bytes to be read from the opened file. This method starts reading from the beginning of the file and if count is missing, then it tries to read as much as possible, maybe until the end of file.

## Example

Let us take a file `foo.txt`, w

# Open a file

```
fo = open("foo.txt", "r+") str = fo.read(10)
```

```
print ("Read String is : ", str)
```

```
# Close opened file fo.close() which we created above.
```

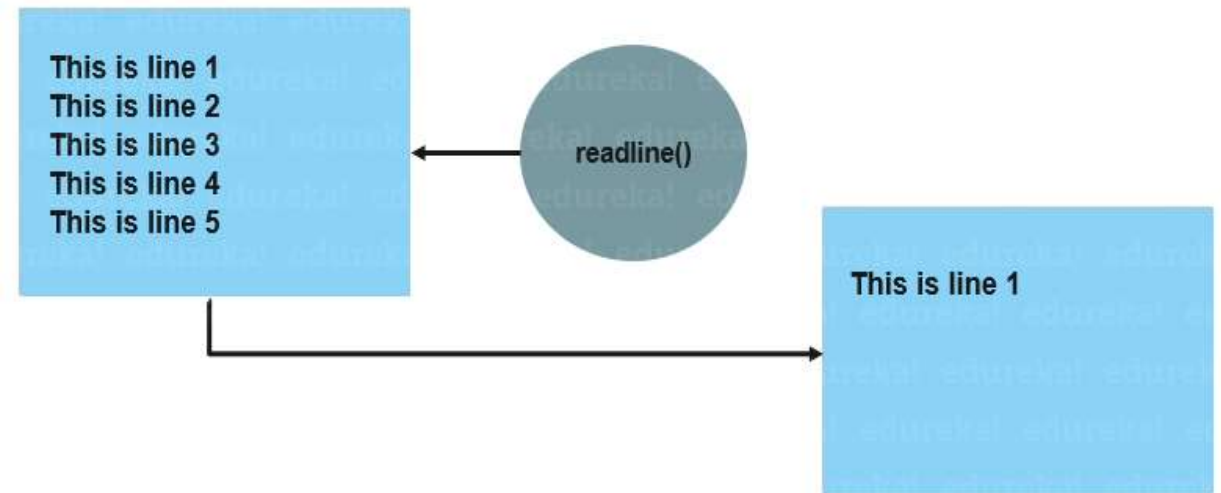


# readline() Method

Python readline() method will return a line from the file when called.

**Syntax:** `f=open('filename.txt','r')`  
`f.readline()`

**Example:** `f=open('filename.txt','r')`  
`line1 = f.readline()`  
`print(line1)`  
`f.close()`

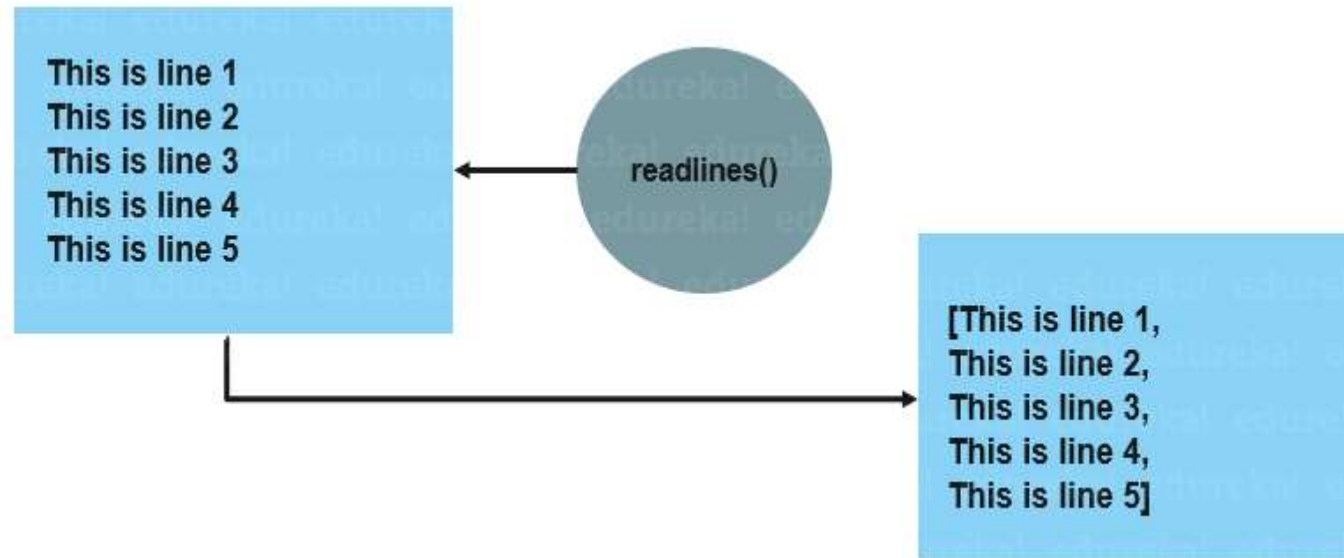


# readlines() Method

`readlines()` method will return all the lines in a file in the format of a list where each element is a line in the file.

**Syntax:** `f=open('filename.txt','r')`  
`f.readlines()`

**Example:** `f=open('filename.txt','r')`  
`example1 = f.readlines()`  
`print(example1)`  
`f.close()`



# Use of for loop to read a file

```
f=open('C:\myfile.txt')
```

```
For line in f:
```

```
    print(line)
```

```
f.close()
```

In the above code 'C:\myfile.txt' is the file name with path.



# Practice Questions

1. Write a Python Program to open a file named data.txt and print its entire content.
2. Write a Python Program to read and print the 1<sup>st</sup> 10 characters of the file sample.txt
3. Write a Python Program to count the number of lines in a file notes.txt
4. Write a Python Program that read the last 5 lines of a text file.
5. Write a Python Program to open a file and display its content its uppercase.





# Writing In Files

# write() Method

The write() function will write the content in the file without adding any extra characters.

**Syntax:** filename.write(content)

**Example:** f=open('filename.txt','w')  
f.write('Hi there, this is the first line of file.\n')  
f.write('and another line.\n')  
f.close()





# writelines() Method

The write() function will write the list of strings in the file.

**Syntax:** filename.writelines(list\_name)

**Example:** lines=['hello world.\n', 'This is another line.\n']  
f=open('filename.txt','w')  
f.writelines(lines)  
f.close()



# Practice Questions

1. Write a program to write multiple lines using multiple `write()` calls.
2. Write lines to a file and then read the content to verify.
3. Write a program that takes user input and saves it to a file using `write()`.
4. Write a program to append lines to an existing file using `write()`.
5. Write a program to copy content from one file to another using `read()` and `write()`.





# Reading and Writing Files using With Statement

# with statement

- with statement will automatically close the file after the nested block of code.
- If an exception occurs before the end of block, it will close the file before the exception is caught by an outer exception handler.
- The advantage of using with statement is that it is guaranteed to close the file.



# Read File using with statement

```
#read entire file as one string  
with open('filename.txt','r') as f:  
    data=f.read()
```

```
#iterate over lines of file  
with open('filename.txt','r') as f:  
    for line in f:  
        print(line, end=' ')
```



# Write in File using with statement

```
#write()
```

```
with open('filename.txt','w') as f:
```

```
    f.write('Hi there, this is the first line of file.\n')
```

```
    f.write('and another line.\n')
```

```
#writelines()
```

```
lines=['hello world.\n', 'This is another line.\n']
```

```
with open('filename.txt','w') as f:
```

```
    f.writelines(lines)
```

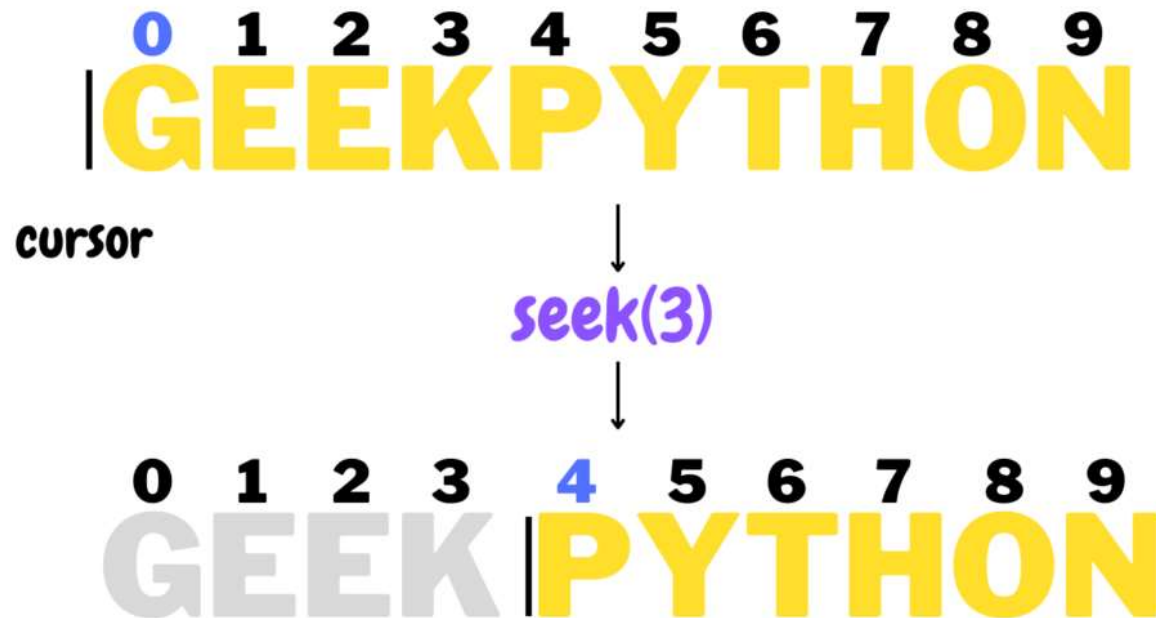




`seek()` and `tell()`

# seek() method

The seek() function in Python is used to move the file cursor to the specified location. When we read a file, the cursor starts at the beginning, but we can move it to a specific position by passing an arbitrary integer (based on the length of the content in the file) to the seek() function.





# seek() method

**Syntax:** seek(offset, whence)

**offset** – Required parameter. Sets the cursor to the specified position and starts reading after that position.

**whence** – Optional parameter. It is used to set the point of reference to start from which place.

0 – Default. Sets the point of reference at the beginning of the file.

1 – Sets the point of reference at the current position of the file.

2 – Sets the point of reference at the end of the file.



# seek() method

# Opening a file for reading

with open('sample.txt', 'r') as file:

# Setting the cursor at 62nd position

file.seek(62)

# Reading the content after the 62nd character

data = file.read()

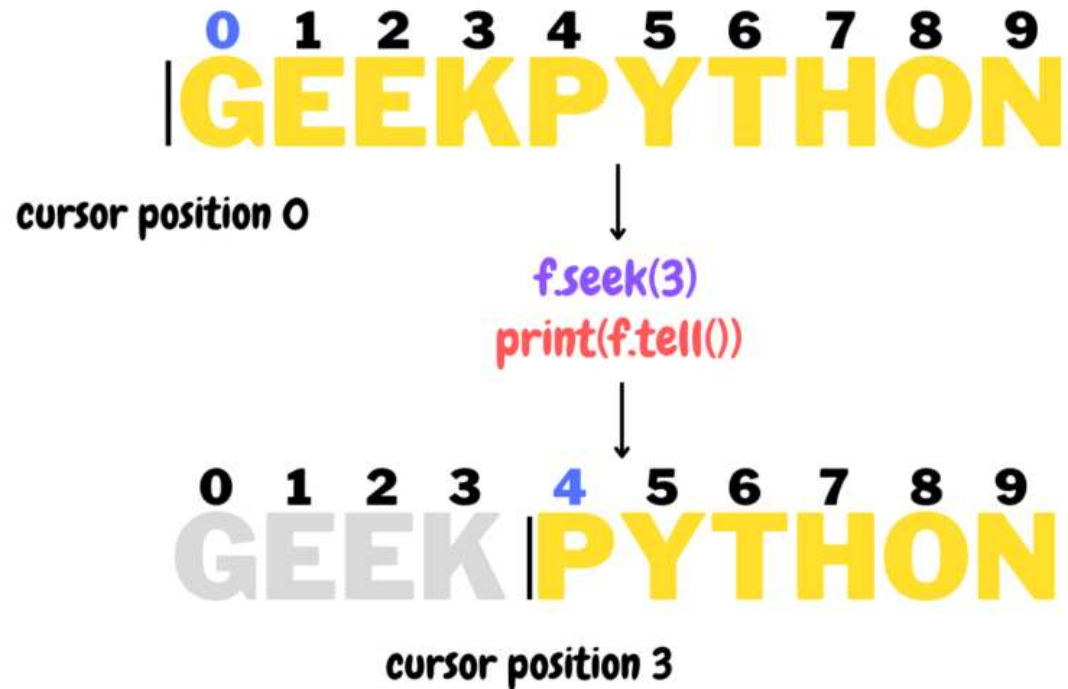
- print(data)

We've moved the cursor to the 62nd position, which means that if we read the file, we'll begin reading after the 62nd character.



# tell() method

tell() function returns the position where the cursor is set to begin reading.



# tell() method

# Opening a file in binary mode

with open('sample.txt', 'r') as file:

# Setting cursor at the 25th pos from the  
 end

file.seek(-25, 2)

# Using tell() function

pos = file.tell()

# Printing the position of the cursor

print(f'File cursor position: {pos}')

# Printing the content

data = file.read()

print(data)

## Output:

File cursor position:127

Content of file



# Practice Questions

1. Write a program to demonstrate moving the file pointer using `seek()`.
2. Write a program that uses `tell()` to print position before and after a `read()`.
3. Write a program to skip the first 5 characters and print the rest using `seek()`.
4. Write a program to use `tell()` to track reading position line by line.
5. Write a program to show the current file pointer position using `tell()`.

